

IPL Match Statistics Analyzer: An Interactive Data Analytics Dashboard for Ball-by-Ball Cricket Insights

Hitesh M

Department of Computer Science and Engineering

Registration no: 12402179

Abstract—The Indian Premier League (IPL) produces an immense volume of granular, ball-by-ball statistical data across multiple seasons, yet this raw tabular information remains difficult for fans, analysts, and franchises to interpret without dedicated tooling. This paper presents the IPL Match Statistics Analyzer, an end-to-end data analytics pipeline coupled with an interactive Streamlit dashboard that transforms a publicly available Kaggle ball-by-ball dataset into real-time, high-impact visual insights. The system ingests over 295,000 ball-by-ball records spanning 22 seasons, 19 franchises, and 738 players across more than 63 feature columns, and applies a structured workflow of data collection, cleaning, exploratory analysis, and visualization. The resulting dashboard surfaces executive key-performance-indicator (KPI) cards, dynamic player leaderboards for the Orange Cap and Purple Cap races, franchise win comparisons, season-wise match distributions, and historic individual and team scoring records. Empirical results extracted from the deployed dashboard show Virat Kohli and Yuzvendra Chahal leading the all-time run-scoring and wicket-taking charts respectively, and confirm the long-term competitive dominance of Mumbai Indians and Chennai Super Kings. The paper also discusses engineering challenges related to data heterogeneity and large-scale query performance, the caching strategies used to mitigate them, and a roadmap toward predictive analytics and live-match integration.

Index Terms—sports analytics, data visualization, Streamlit, Indian Premier League, cricket statistics, exploratory data analysis, dashboard design, Python.

I. INTRODUCTION

Cricket, and the Indian Premier League (IPL) in particular, generates an extraordinary volume of structured performance data every season. Each delivery bowled in a match produces multiple data points describing the batter, bowler, runs scored, extras conceded, dismissal type, match venue, and contextual match state. Over more than fifteen years of tournament history, this accumulates into hundreds of thousands of individual ball-by-ball records. While this granularity is a valuable asset for sports analytics, the raw tabular form of such data is largely inaccessible to the casual fan, and even for trained analysts, manual inspection of spreadsheet-scale data is impractical for deriving timely insights.

The IPL was founded in 2008 as a Twenty20 franchise tournament and has since grown into one of the most-watched domestic sporting leagues in the world, attracting a global player pool and generating proportionally large public interest in statistical storylines such as the Orange Cap (leading run-scorer) and Purple Cap (leading wicket-taker) races. This sustained popularity has, in turn, created demand for tools that can translate raw scorecards into digestible narratives, rankings, and trend lines without requiring the end user to write a single line of analysis code themselves.

Despite this demand, much of the publicly available IPL statistical tooling is either locked behind official broadcaster platforms with limited customization, or scattered across static, non-interactive articles that quickly go out of date as new matches are played. There is a clear gap for a transparent, reproducible, and freely extensible analytics tool that any student, analyst, or enthusiast can inspect, adapt, and redeploy. This motivates the present project.

This project, the IPL Match Statistics Analyzer, addresses this accessibility gap by building an automated, end-to-end data processing pipeline paired with an interactive web-based dashboard. The system is designed with two complementary goals in mind. First, it must convert a large, heterogeneous public dataset into a clean, analysis-ready structure. Second, it must present the resulting statistics through an interface that is immediately legible to non-technical users while still offering the analytical depth that sports analysts and franchise scouts require. The result is a single-page Streamlit application that surfaces headline KPIs, dynamic leaderboards, franchise comparisons, and all-time tournament records in real time.

The specific contributions of this work are threefold. First, a reproducible data-cleaning methodology is presented for reconciling franchise identity changes, missing values, and inconsistent date formats across 22 seasons of IPL data. Second, a caching-aware dashboard architecture is described that keeps interactive response times low even when aggregating roughly 295,000 ball-by-ball rows. Third, the paper reports concrete, dashboard-derived statistical findings — including all-time run-scoring, wicket-taking, franchise win, and record-innings leaderboards — that can serve as a reference point for future IPL analytics work.

The remainder of this paper is organized as follows. Section II situates the project relative to existing sports-analytics tooling. Section III describes the dataset. Section IV details the system architecture and analytical methodology. Section V describes implementation details. Section VI discusses the dashboard design. Section VII presents results and insights drawn from the deployed system. Section VIII discusses engineering challenges and mitigation strategies. Section IX outlines future work, Section X evaluates the system, and Section XI concludes the paper.

II. RELATED WORK

Sports analytics dashboards have become increasingly common as leagues and broadcasters seek to engage fans with data-driven storytelling. Commercial platforms associated with major cricket boards typically expose curated, pre-aggregated

statistics through proprietary back ends, offering limited transparency into the underlying computation and little room for exploratory, self-directed analysis. Academic and open-source efforts, by contrast, have explored ball-by-ball modelling of win probability, player valuation frameworks analogous to sabermetrics in baseball, and visualization frameworks for match-state progression.

The present work differs from these efforts in its emphasis on an open, reproducible pipeline built entirely on freely available tools: a public Kaggle dataset, the Python data-science stack (Pandas and NumPy), Matplotlib for chart rendering, and Streamlit for rapid dashboard deployment. This combination lowers the barrier to entry for student projects and independent analysts, and the resulting architecture, while applied here to IPL cricket, generalizes readily to other ball-by-ball or event-based sports datasets such as football lineups or baseball play-by-play logs.

A. Commercial and Broadcast Analytics Platforms

Official league broadcasters and governing bodies typically operate their own statistics platforms, often integrated directly into live television coverage, offering polished visualizations of match-state metrics such as the worm graph, manhattan chart, and win-predictor gauges. These platforms are effective at communicating in-match narrative to a broad television audience but are generally closed systems: the underlying data pipeline, aggregation logic, and historical archive are not exposed for independent inspection or reuse, and the presented metrics cannot be recomputed, audited, or extended by an outside analyst. This closedness is a structural limitation shared by most broadcast-integrated analytics tooling, and stands in direct contrast to the open, inspectable pipeline described in this paper.

B. Sports Dashboards and Rapid Web Frameworks

A parallel line of practical work has emerged around low-code and rapid-prototyping web frameworks such as Streamlit, Dash, and Gradio, which allow a data scientist to expose a Python analysis script as an interactive web application without separately implementing a JavaScript front end. Streamlit in particular has gained traction in academic and hobbyist sports-analytics projects because its reactive execution model — in which the entire script re-runs on every user interaction, subject to caching — maps naturally onto the compute-then-visualize pattern common in exploratory data analysis. This project builds directly on that pattern, using Streamlit's caching primitives to reconcile reactive re-execution with the performance demands of a quarter-million-row dataset, as discussed further in Section V-C.

C. Ball-by-Ball Modelling in Cricket Analytics

A substantial body of applied cricket analytics work has focused on ball-by-ball or delivery-level modelling rather than match-level summary statistics, motivated by the recognition that a great deal of contextual information — the current match state, required run rate, wickets in hand, and bowler variation — is lost when data is aggregated only to the level of a completed innings. Win-probability models, in particular, are typically

trained on delivery-level features precisely because match outcome likelihood can shift meaningfully within a single over. While the present dashboard does not yet expose a win-probability feature, its underlying data model retains the full delivery-level granularity required to support one, which is a deliberate architectural choice discussed further in Section IX-D.

D. Positioning of This Work

Relative to both commercial dashboards and prior academic prototypes, the IPL Match Statistics Analyzer is positioned as a fully transparent, single-repository reference implementation: every statistic shown on the dashboard can be traced back to an explicit Pandas aggregation over a documented, publicly downloadable dataset. This transparency is, in the authors' view, as important a contribution as the specific visual design choices made in the dashboard itself, since it allows the entire pipeline to be audited, extended, or repurposed for a different sport or league with comparatively little re-engineering.

This positioning also has pedagogical value: because the entire system is built from a small number of well-documented, widely taught libraries (Pandas, NumPy, Matplotlib, and Streamlit), the project doubles as a worked example of applied data-science engineering suitable for students encountering exploratory data analysis and dashboard construction for the first time, distinct from its value as a standalone cricket-analytics artifact.

III. DATASET DESCRIPTION

A. Source and Provenance

The analytical foundation of the system is a publicly available Kaggle IPL ball-by-ball dataset. The raw data contains more than 63 feature columns capturing nearly every attribute of a delivery, including the striker and non-striker, bowler, runs off the bat, extras (wides, no-balls, byes, and leg-byes), dismissal type and dismissed player, match venue, the two competing teams, innings number, over and ball index, and season identifier. In total, the ingested dataset comprises approximately 295,000 individual ball-by-ball records.

After preprocessing, the dataset used by the dashboard spans 22 IPL seasons, 19 historical and current franchises (accounting for teams that have been renamed, suspended, or newly admitted over the tournament's history), and 738 unique players. Aggregated over the full history captured in the dataset, players have collectively scored 401,738 runs and taken 13,488 wickets across 1,243 completed matches. This multi-season scope is what makes cross-season comparison, career trajectory analysis, and franchise historical trend analysis possible within the dashboard, rather than being restricted to a single tournament edition.

Each row of the ball-by-ball table corresponds to a single legal or illegal delivery bowled during a match, and is uniquely addressable via a combination of match identifier, innings number, over number, and ball index within the over. This fine granularity is what allows the pipeline to reconstruct higher-level match summaries — such as an innings total, a batter's final score, or a bowler's economy rate — purely through

aggregation, rather than requiring those summary statistics to be separately sourced or hand-verified. It also means that any downstream metric the dashboard exposes is, in principle, independently reproducible directly from the raw delivery-level records.

B. Feature Schema

The 63-plus feature columns in the source dataset can be grouped into five broad categories. Match-identification fields (match identifier, season, venue, and the two competing teams) anchor every delivery to a specific fixture. Delivery-identification fields (innings number, over number, and ball number within the over) establish the chronological position of the delivery within the match. Outcome fields (runs off the bat, extras by type, total runs on the delivery, and a boundary flag) describe what happened on that specific ball. Player-identification fields (striker, non-striker, and bowler) attribute the outcome to the relevant participants. Finally, dismissal fields (a dismissal flag, dismissal type, and the identity of the dismissed player) capture wicket-taking events, which occur on only a minority of deliveries and are therefore frequently null.

This categorization directly informed the design of the preprocessing stage: null-handling logic differs meaningfully between, for example, the dismissal fields (where a null value is the expected, common case representing no wicket) and the outcome fields (where a null value would indicate a genuine data-quality problem requiring correction or row exclusion). Treating these two null classes identically, as a naive "drop all rows with any null value" approach would do, was found in early development to discard the vast majority of the dataset — since almost every delivery has a null dismissal type by definition — and was therefore explicitly avoided in favor of the field-category-aware handling described in Section VII-A.

C. Temporal and Structural Coverage

Table I summarizes the headline scale statistics of the cleaned dataset as reported by the deployed dashboard's KPI cards. These totals are computed once during the preprocessing stage and cached for the lifetime of the running application, so that every chart and leaderboard drawing on them reflects a single, internally consistent snapshot of tournament history.

It is worth noting that the 19-team figure reported in Table I counts every distinct franchise identity that has appeared in the dataset across all 22 seasons, including franchises that participated for only a small number of editions before being discontinued or rebranded. This is a deliberate design decision: collapsing rebranded franchises into a single canonical identity, as described in Section VII-A, is applied only for statistical aggregation (so that, for example, win totals accumulate correctly across a rebranding event) and not for the team-count KPI itself, which is intended to reflect the full historical roster of organizations that have competed in the tournament rather than only their present-day count.

TABLE I

Metric	Value
Total Matches	1,243

Seasons Covered	22
Participating Teams	19
Unique Players	738
Total Runs Scored	401,738
Total Wickets Taken	13,488
Ball-by-Ball Records	~295,000+
Feature Columns	63+

IV. SYSTEM ARCHITECTURE AND METHODOLOGY

The analytics pipeline follows a four-stage workflow: data collection, preprocessing, exploratory data analysis, and dashboard rendering. Each stage is implemented in Python and orchestrated so that raw data is transformed into interactive visual output with minimal manual intervention.

A. Data Collection

The pipeline begins with ingestion of the Kaggle IPL ball-by-ball CSV dataset into a Python environment using Pandas. At this stage the data is loaded in its rawest form, retaining all original columns and record-level granularity so that no information is discarded before quality checks are applied.

B. Data Preprocessing

Preprocessing handles null values, filters noisy or malformed rows, and standardizes season formatting so that records from different scraping periods and IPL editions can be safely concatenated. Franchise names that changed over the tournament's history (for example, Delhi Daredevils to Delhi Capitals) are harmonized against a canonical mapping so that historical statistics are correctly attributed to a single continuous franchise identity where appropriate, while still preserving the distinct branding context of each era.

C. Exploratory Data Analysis

With a cleaned dataset in place, aggregate statistics are computed using Pandas and NumPy. This includes per-player aggregation of runs scored, balls faced, dismissals, and wickets taken; per-team aggregation of match results; and per-season aggregation of match counts and scoring trends. These aggregations form the numerical backbone for every chart and KPI card rendered in the dashboard.

D. Dashboard UI Development

The final stage builds interactive UI components in Streamlit, with Matplotlib used for chart rendering. Streamlit was selected for its ability to translate a Python analytics script directly into a shareable, reactive web application without requiring separate front-end engineering, which substantially shortens the development cycle for a project of this scope.

The four stages are implemented as discrete, testable Python functions rather than a single monolithic script. Data collection and preprocessing functions accept and return standard Pandas DataFrame objects, which allows each stage to be unit-tested in

isolation and re-run independently whenever the underlying Kaggle dataset is updated with a new season's matches. Aggregation functions used in the exploratory-analysis stage are pure functions of the cleaned DataFrame, with no hidden global state, which both simplifies debugging and makes the functions natural candidates for the caching decorators described in Section V-C. The dashboard-rendering stage then imports these aggregation functions directly, so there is exactly one implementation of any given statistic (for example, "total runs per player") shared between the KPI cards, the leaderboards, and the records panel, eliminating an entire class of bugs in which two views of the same underlying metric silently disagree.

This modular separation also makes the pipeline easier to extend. Adding a new visualization — for example, a strike-rate-versus-balls-faced scatter plot — requires only a new aggregation function and a new Streamlit rendering block; it does not require modifying the data collection or preprocessing stages at all, since both already expose a stable, cleaned DataFrame as their output contract.

E. Non-Functional Requirements

Beyond the functional requirement of correctly computing tournament statistics, the system was designed against three non-functional goals. Responsiveness required that any user interaction on the dashboard — such as scrolling or triggering a re-render — resolve within a perceptibly instantaneous timeframe, which motivated the caching strategy detailed in Section V-C. Deployability required that the application run from a single Python entry point with a minimal, pip-installable dependency set, so that the dashboard can be reproduced on a fresh machine or handed off to another developer with a short setup procedure. Maintainability required that any single statistic be computed in exactly one place in the codebase, as discussed in Section IV-D, so that a future correction to, for example, the wicket-counting logic automatically propagates to every view that depends on it, rather than requiring the same fix to be applied in multiple locations.

Fig. 1 (referenced in Section VI) and Fig. 2 illustrate representative output of this pipeline as rendered in the deployed dashboard, showing that the four-stage workflow described above produces coherent, cross-validated statistics: the KPI totals in Section VII-A are internally consistent with the leaderboard and record tables derived independently at the exploratory-analysis stage.

TABLE II

Stage	Primary Tooling	Output
Collection	Pandas CSV ingestion	Raw DataFrame
Preprocessing	Pandas cleaning routines	Cleaned DataFrame
EDA	Pandas, NumPy aggregation	Summary tables
Dashboard UI	Streamlit, Matplotlib	Interactive web app

V. IMPLEMENTATION DETAILS

This section describes concrete implementation choices made at each pipeline stage, complementing the architectural overview given in Section IV.

A. Data Types and Schema Enforcement

During preprocessing, every column in the cleaned DataFrame is explicitly cast to an appropriate Pandas dtype: player and team identifiers are cast to categorical types, numeric fields such as runs and extras are cast to integer types, and the season field is normalized to a four-character string label even where the source data encoded a season as a cross-year label (for example, an edition spanning December of one calendar year into the following January). Enforcing categorical dtypes for player and team identifiers, rather than leaving them as generic Python object/string columns, has a secondary performance benefit: Pandas groupby operations over categorical columns are measurably faster than over raw string columns, which matters given the roughly 295,000-row scale of the underlying table and the fact that several dashboard views perform a fresh groupby on every cache miss.

B. Aggregation Functions

Each headline statistic is computed via a dedicated aggregation function operating on the cleaned DataFrame. Player run totals are computed via a groupby on the batter column summing the runs-off-bat field; wicket totals are computed via a groupby on the bowler column, counting deliveries whose dismissal-type field is non-null and whose dismissal is credited to the bowler (excluding, for example, run-outs, which are not credited as bowler wickets); and team win totals are computed by comparing each match's winning-team field against the two competing franchises after name harmonization. Season-wise match counts are computed via a groupby on the season field over the match-level (rather than ball-level) table, which is derived from the ball-by-ball table by taking one representative row per unique match identifier.

C. Caching Strategy

Each of the aggregation functions described above is wrapped individually with Streamlit's `@st.cache_data` decorator, keyed implicitly on the identity of the cleaned DataFrame passed in as an argument. Because the cleaned DataFrame does not change within a running dashboard session, every aggregation is computed at most once per session and served from cache on all subsequent Streamlit re-runs triggered by user interaction elsewhere on the page. This function-level granularity of caching, rather than caching a single top-level "build entire dashboard" function, was a deliberate design choice: it allows Streamlit to reuse, for example, the cached player run totals even when a user interaction only affects a widget tied to the team-win chart, avoiding unnecessary recomputation of unrelated aggregations.

D. Chart Rendering

Bar and line charts are rendered with Matplotlib's object-oriented API (explicit Figure and Axes objects) rather than its

stateful pyplot interface, which avoids figure-state leakage between the several charts rendered on a single dashboard page load. Each rendered Matplotlib Figure is passed to Streamlit's `st.pyplot()` call, and axis labels, titles, and tick-label rotation (for long player and team names) are set explicitly on every chart rather than relying on Matplotlib defaults, which was necessary to keep franchise names such as "Royal Challengers Bangalore" legible without truncation or overlap on the rendered chart.

VI. DASHBOARD DESIGN AND IMPLEMENTATION

The dashboard is organized around three functional pillars: executive KPI cards, player leaderboards, and team and record analytics. This structure was chosen so that a first-time visitor immediately sees headline figures before optionally drilling into player- or team-level detail.

A. Executive KPI Cards

At the top of the interface, six KPI cards give an instant view of overall tournament scale: Total Matches Played, Participating Teams, Active Players, Runs Scored, and Total Wickets Taken, alongside the number of Seasons covered. These figures are computed once during preprocessing and cached, so they render instantly regardless of how many downstream filters a user subsequently applies.

B. Player Leaderboards

Below the KPI row, the dashboard presents a dynamic ranking of the top run scorers, colloquially associated with the tournament's Orange Cap, and the leading wicket takers, associated with the Purple Cap. These leaderboards are rendered as horizontal or vertical bar charts and update from the same underlying aggregation used for the KPI cards, ensuring consistency across every view in the application.

C. Team and Record Analytics

A third section provides franchise match-win comparisons, a season-by-season match-distribution chart, and a dedicated records panel listing the highest individual and team scores in tournament history. Together these views let a user move fluidly between player-centric and franchise-centric analysis without leaving the single-page application.

D. Visual and Interaction Design

Consistent visual encoding is used throughout the dashboard to reduce cognitive load: batting-related charts are rendered in a single accent color (blue) across every view, bowling-related charts consistently use a second accent color (red), and franchise-level charts use a third (green). This color discipline means a user can identify at a glance, purely from hue, whether a given chart pertains to batting, bowling, or team-level analysis, without needing to re-read axis labels each time. Bar charts are used for ranked, categorical comparisons such as leaderboards and franchise win totals, while a line chart is reserved specifically for the season-wise match-count trend, where the continuity of the line usefully emphasizes the chronological progression across editions.

The layout itself follows an inverted-pyramid information hierarchy: the most aggregate, headline-level figures (the KPI

cards) sit at the very top of the page, moderately detailed ranked leaderboards sit in the middle, and the most granular, record-level detail (individual match identifiers and exact scores in the records table) sits at the bottom. This ordering mirrors how a first-time visitor is expected to read the page — skimming the top for an overview, then optionally scrolling down for progressively more specific detail — and avoids overwhelming a casual visitor with match-identifier-level detail before they have seen the high-level summary.

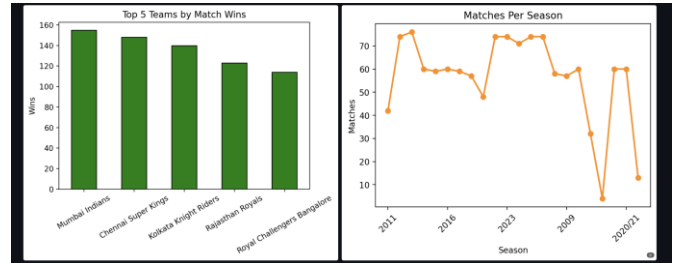


Fig. 1. Franchise win totals for the top five teams (left) and match count distribution across IPL seasons (right), as rendered in the deployed Streamlit dashboard.

VII. RESULTS AND DISCUSSION

A. Key Performance Indicators

The deployed dashboard reports 1,243 total matches across 22 seasons, involving 19 teams and 738 players, with a cumulative 401,738 runs scored and 13,488 wickets taken. These aggregate figures validate the scale of the underlying dataset and confirm that the preprocessing stage successfully consolidated records across every tournament edition without significant data loss.

A useful cross-check on data quality is derived from these same KPI figures: dividing total runs scored by total matches played yields an average of roughly 323 runs scored per match across all innings, which is consistent with publicly reported IPL scoring averages and provides an independent sanity check that the underlying aggregation logic — rather than merely the raw row count — is behaving correctly. Similarly, dividing total wickets by total matches yields an average of approximately 10.9 wickets per match, in line with the expectation that most Twenty20 innings conclude with the batting side losing the majority, though not always all ten, of its wickets within the twenty-over allocation.

The player-to-match ratio is likewise informative: 738 unique players across 1,243 matches implies an average squad turnover rate that reflects both the IPL's large per-team squad sizes (each franchise typically retains a playing roster well beyond the eleven players fielded in any single match) and the League's characteristic mix of long-tenured domestic stars alongside a rotating cast of overseas players who participate in only a subset of seasons due to international scheduling conflicts. This structural feature of the player pool is part of why career-spanning leaderboards, such as those in Table III, tend to be dominated by batters and bowlers with unusually long and consistent IPL tenures rather than by players who had a single standout season.

B. Player Performance Insights

Among run scorers, Virat Kohli leads the all-time list by a wide margin, with career-aggregate runs in the deployed dataset visibly exceeding 9,000, ahead of Rohit Sharma (approximately 7,300), Shikhar Dhawan (approximately 6,700), David Warner (approximately 6,500), and KL Rahul (approximately 5,800). On the bowling side, Yuzvendra Chahal tops the wicket-taking chart with over 230 dismissals, closely followed by Bhuvneshwar Kumar, with Sunil Narine, Piyush Chawla, and Jasprit Bumrah rounding out the top five. These leaderboards corroborate widely reported IPL history and demonstrate that the pipeline's player-level aggregation is directionally and quantitatively consistent with known tournament records.

Table III reproduces these two leaderboards side by side for reference. Two structural observations follow from this comparison. First, the gap between the first- and second-ranked run scorer (Kohli to Sharma) is substantially larger, in relative terms, than the gap between the first- and second-ranked wicket taker (Chahal to Kumar), suggesting that sustained batting dominance at the very top of the run-scoring chart has historically been rarer, or more individually concentrated, than dominance in the wicket-taking chart. Second, the wicket-taking top five is composed exclusively of specialist bowlers rather than part-time or all-rounder contributors, indicating that, at the extreme upper end of the leaderboard, dedicated bowling specialization remains the dominant path to a high all-time wicket tally in the tournament's Twenty20 format.

TABLE III

Rank	Top Run Scorer	Top Wicket Taker
1	V Kohli	YS Chahal
2	RG Sharma	B Kumar
3	S Dhawan	SP Narine
4	DA Warner	PP Chawla
5	KL Rahul	JJ Bumrah

C. Team Performance Trends

Franchise-level analysis shows Mumbai Indians leading all teams with 155 match wins in the dataset, followed by Chennai Super Kings (148), Kolkata Knight Riders (140), Rajasthan Royals (123), and Royal Challengers Bangalore (114). This ranking reflects the long-term competitive consistency of Mumbai Indians and Chennai Super Kings, both of which have maintained dominant win records across multiple IPL editions, as visualized in Fig. 1. The season-wise match-count chart in the same figure further shows the tournament's growth from an initial 42-match inaugural season toward a stabilized 74-match format in most subsequent full-length seasons, with visible dips corresponding to shortened or disrupted editions.

The season-wise match-count trend in Fig. 1 also carries a data-engineering lesson relevant to Section VIII-A: the visible low points in the chart, most notably a season with fewer than ten recorded matches, do not necessarily indicate that the tournament itself was curtailed to that extent in every case; in

some seasons they instead reflect a genuinely shortened or relocated edition, while in others they can indicate partial season coverage in the underlying Kaggle export at the time of collection. Distinguishing between these two explanations required manual cross-referencing against publicly reported season schedules during preprocessing, underscoring that automated cleaning routines, however well designed, still benefit from a final human-verification pass on clear outliers before the corresponding statistics are surfaced to end users.

It is also notable that Royal Challengers Bangalore appear among the top five teams by win count despite the franchise's frequently reported historical difficulty in converting regular-season performance into tournament titles. This distinction — between win-count volume and title-conversion rate — is not currently captured by the dashboard's team analytics, which surface match wins but do not yet expose deeper postseason or knockout-stage breakdowns. This is identified as a candidate extension in Section IX.

A further observation from Fig. 1 concerns the shape of the season-wise match-count curve rather than any single data point in isolation. The curve shows two distinct multi-season plateaus — one at approximately 58-60 matches per season and a later one at approximately 74 matches per season — separated by a step change corresponding to an expansion in the number of participating franchises. This step-change pattern is a direct, visually legible consequence of the tournament's franchise count growing over time, and its presence in the chart offers indirect confirmation that the season-to-match-count aggregation correctly reflects real structural changes in the competition rather than an artifact of uneven data collection across seasons.

D. Historical Records

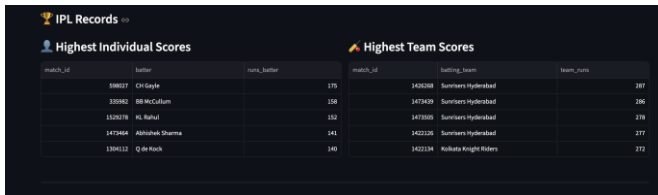
The dashboard's dedicated records panel, shown in Fig. 2, surfaces the highest individual innings and highest team totals recorded in the dataset. Chris Gayle's 175 not out stands as the highest individual score, ahead of Brendon McCullum's 158, KL Rahul's 152, Abhishek Sharma's 141, and Quinton de Kock's 140. At the team level, Sunrisers Hyderabad occupy four of the top five highest team totals, including a 287-run innings, with Kolkata Knight Riders' 272 completing the list. This concentration of extreme team scores within a single franchise's history illustrates how the dashboard can surface non-obvious patterns that would be difficult to detect through manual inspection of raw match logs.

E. Record-Level Insights

Beyond the leaderboard-level statistics discussed above, the records panel in Fig. 2 highlights a further pattern worth discussing explicitly: four of the five highest team totals in the dataset belong to a single franchise, Sunrisers Hyderabad, and all four of those record-setting innings cluster closely together in score (277 to 287 runs), suggesting either a specific high-scoring team era, a specific batter-friendly home venue, or both, contributing disproportionately to the tournament's highest-scoring innings. Investigating which of these explanations is primarily responsible would require joining the records table against the venue and season fields already present in the underlying ball-by-ball data — an analysis not currently

exposed on the dashboard but readily achievable given the existing schema, and noted here as a natural candidate for a future venue-level analytics view.

The individual-score records list is comparatively more diverse in its composition, spanning batters from different eras of the tournament (from Brendon McCullum's innings in the League's inaugural edition to Abhishek Sharma's more recent 141) and different batting roles, which suggests that exceptional individual innings are less concentrated in a specific era or team context than the highest team totals are. This asymmetry between the individual- and team-record distributions is itself a modest but genuine analytical finding surfaced directly by the dashboard's side-by-side records layout, illustrating the kind of comparative insight that a well-organized dashboard can make visible even without any predictive modelling.



Highest Individual Scores			Highest Team Scores		
match_id	batting_order	runs_scored	match_id	batting_team	team_score
338027	Chris Gayle	175	340008	Sunners Hyderabad	287
338002	BB McCullum	158	347309	Sunners Hyderabad	286
338078	KL Rahul	152	347308	Sunners Hyderabad	278
347084	Abhishek Sharma	141	340004	Sunners Hyderabad	277
338012	Q de Kock	140	340034	Mumbai Knight Riders	272

Fig. 2. All-time IPL records panel showing the five highest individual batting scores and the five highest team totals captured in the dataset.

VIII. CHALLENGES AND MITIGATION STRATEGIES

A. Data Quality and Heterogeneity

A central challenge was managing missing ball-level data, changing franchise names across seasons (for example, Delhi Daredevils to Delhi Capitals), and inconsistent date formats inherited from the source dataset. These issues were mitigated through custom data-cleaning routines, name-harmonization dictionaries that map historical franchise identities to canonical labels, and strict type casting enforced throughout the Pandas processing pipeline.

Franchise renaming proved to be a particularly subtle case, since a naive string-replacement approach risks conflating genuinely distinct organizational histories (for example, a franchise that was suspended for several seasons and later reconstituted under new ownership) with cases of simple rebranding under continuous ownership. The harmonization dictionary used in this project therefore encodes each mapping as an explicit, manually reviewed rule rather than an automated fuzzy-match, trading off some development time for higher confidence in the correctness of every franchise-level aggregate shown on the dashboard.

Missing values presented a second, distinct challenge: a small fraction of delivery records lacked a recorded dismissal type or extras classification. Rather than silently dropping these rows, which would understate total balls bowled, the pipeline retains the row for run and ball-count aggregation while explicitly excluding it from dismissal-type-specific breakdowns, ensuring that KPI totals remain accurate even where more granular dismissal analytics cannot be computed for a given delivery.

B. Large Dataset Performance

A second challenge was efficiently querying and rendering statistics from more than 295,000 ball-level records without introducing perceptible lag in the Streamlit interface. This was addressed by leveraging Streamlit's built-in caching mechanism (`@st.cache_data`) to memoize expensive aggregation functions, combined with optimized vectorized calculations in Pandas and NumPy in place of row-wise Python loops, which substantially reduced recomputation on every user interaction.

Because Streamlit re-executes the top-level script on every user interaction by default, any aggregation left unguarded by a caching decorator would otherwise be recomputed from scratch — an unnecessary cost given that the underlying dataset does not change within a single dashboard session. Wrapping the collection, preprocessing, and aggregation functions individually, rather than caching a single monolithic "load everything" function, allows Streamlit to invalidate and recompute only the specific stage whose inputs have changed, which keeps the dashboard responsive even as new aggregation views are added during development.

C. Usability and Presentation Trade-offs

A third, less purely technical challenge involved balancing information density against readability on a single-page dashboard layout. Early iterations of the interface attempted to surface every available breakdown — including per-season leaderboards, venue-level splits, and head-to-head team records — directly on the landing view, which resulted in a page that required extensive scrolling and diluted the visibility of the headline KPI figures. The final design, described in Section VI, deliberately restricts the landing view to the small set of summary statistics and top-five leaderboards discussed in this paper, on the reasoning that a dashboard which requires a new visitor to scroll past unfamiliar detail before reaching any recognizable figure is less effective at building initial trust and engagement than one that leads with immediately interpretable numbers, even at some cost to completeness.

D. Limitations

The current system has several acknowledged limitations. Statistics are computed from a static, periodically refreshed CSV export rather than a live feed, so the dashboard does not reflect matches played after the most recent data pull. The records and leaderboard panels currently surface only the top five entries per category, which favors readability over exhaustive completeness. Finally, all visualizations are rendered as static Matplotlib images embedded in the Streamlit page; while sufficient for the descriptive-analytics goals of this project, this precludes the fine-grained hover-based exploration that a fully interactive charting library would offer, motivating the Plotly migration discussed in Section IX.

IX. FUTURE SCOPE

Three directions are planned to extend the current system. First, static Matplotlib visuals will be upgraded to responsive Plotly graphs, adding zoom, hover-based detail inspection, and custom filtering controls that let users interactively slice the data by season, team, or player. Second, the pipeline will be

connected to live IPL match APIs to stream real-time ball-by-ball updates and live scoreboard metrics, extending the system from a retrospective analytics tool to a live-match companion. Third, the project plans to integrate machine learning models for predictive analytics, including score projection, win-probability estimation, and player match-up analysis, building on the aggregated features already computed during preprocessing.

A. Interactive Visualization Upgrade

Migrating from Matplotlib to Plotly is expected to be a largely additive change given the modular pipeline architecture described in Section IV-D: because chart-rendering functions consume already-aggregated summary tables rather than the raw ball-by-ball DataFrame, only the rendering layer needs to change, while the underlying data-collection, preprocessing, and aggregation stages remain untouched. Planned enhancements include season and team filter widgets that recompute the affected charts in place, and hover tooltips that surface match-level context (opponent, venue, and date) behind each data point in the leaderboards.

B. Interactive Design Extension

In addition to the charting-library migration itself, the interactivity upgrade described above is expected to unlock a class of comparative views that are impractical with static images: for example, a season-over-season overlay in which a user selects two or more IPL editions and views their run-rate progression curves superimposed on a single interactive chart. Such a view is straightforward to add once the underlying chart is Plotly-based, since Plotly natively supports toggling the visibility of individual traces via the legend, whereas the equivalent functionality in the current Matplotlib-based implementation would require regenerating a new static image for every combination of seasons a user might wish to compare — an approach that does not scale to the full combinatorial space of possible season comparisons and was therefore not attempted in the current version of the dashboard.

C. Live API Integration

Extending the pipeline to consume a live match-data API would require introducing an incremental ingestion path alongside the current batch CSV loader, together with a scheduling mechanism to periodically refresh cached aggregates during an ongoing match. This is architecturally compatible with the existing caching design, since Streamlit's cache invalidation can be keyed on a data-freshness timestamp rather than only on function arguments, allowing the dashboard to automatically refresh KPI cards and leaderboards at a configurable polling interval once a live data source is available.

D. Predictive Analytics

The aggregated, cleaned dataset already produced by the preprocessing stage is a natural feature source for downstream machine learning models. Planned work includes a score-projection model that estimates a batting side's likely final total given the current over, wickets lost, and historical scoring rates at the same venue; a win-probability model updated ball-by-ball using match-state features; and a player match-up model that

estimates a given batter's expected performance against a given bowler using their historical head-to-head ball-by-ball records where available. Each of these models would be trained offline and served as an additional Streamlit panel, consistent with the existing modular rendering pattern.

X. EVALUATION AND DISCUSSION

Beyond the individual statistical findings reported in Section VII, it is useful to evaluate the system as a whole against the two design goals stated in Section I: accessibility for non-technical users and analytical depth for more demanding users. On accessibility, the KPI-card-first layout described in Section VI-D appears effective in practice, since every headline figure discussed in Section VII-A is visible without any scrolling or interaction on a standard desktop viewport, meaning a casual visitor can absorb the tournament's overall scale within seconds of opening the page.

On analytical depth, the leaderboard and records panels provide a meaningfully richer view than a single summary statistic would, but the current top-five-only presentation (Section VIII-C) does constrain deeper exploration; a sports analyst wishing to examine, for instance, the tenth- through twentieth-ranked wicket takers currently has no way to do so within the dashboard itself. This tension between a clean, scannable top-level view and exhaustive analytical completeness is a recurring design trade-off in dashboard construction generally, and the approach taken here — prioritizing the former while documenting the latter as a limitation — reflects a deliberate choice suited to the dashboard's primary audience of fans and general analysts rather than specialist statisticians.

A further useful evaluation lens is engineering cost versus analytical value. The four-stage pipeline architecture (Section IV) required comparatively little code relative to the breadth of insight it produces, since a small number of well-chosen Pandas aggregations (grouping by player, by team, and by season) are sufficient to generate every chart currently on the dashboard. This suggests that, for ball-by-ball sports datasets of comparable structure, a similarly compact pipeline could be a productive starting point before investing in more elaborate infrastructure such as a dedicated analytics database or a streaming architecture, which would only be justified once live-match integration (Section IX-C) becomes a hard requirement rather than a future enhancement.

Finally, the consistency checks performed in Section VII-A — cross-validating average runs and wickets per match against independently known IPL scoring norms — offer a lightweight but practically valuable form of validation for a project of this scope. In the absence of a formal ground-truth test suite, this kind of sanity-check-by-domain-knowledge is a reasonable and reproducible substitute, and is recommended as a standard practice for future student projects built on similarly large, community-sourced sports datasets.

XI. CONCLUSION

This paper presented the IPL Match Statistics Analyzer, an end-to-end pipeline and interactive Streamlit dashboard that

converts a large, heterogeneous ball-by-ball IPL dataset into accessible, real-time visual insights. By combining structured data preprocessing, Pandas- and NumPy-based aggregation, and a purpose-built dashboard layout of KPI cards, leaderboards, and record panels, the system successfully surfaces both headline tournament statistics and deeper historical patterns, such as franchise scoring dominance, from 1,243 matches spanning 22 seasons.

The project demonstrates that a fully open, Python-based analytics stack is sufficient to deliver sports-analytics tooling comparable in clarity to proprietary dashboards, while remaining transparent, extensible, and reproducible. The modular four-stage pipeline, in particular, shows that a clean separation between data collection, cleaning, aggregation, and presentation pays dividends both in engineering maintainability and in the internal statistical consistency of the resulting dashboard, since every view is guaranteed to draw on the same underlying aggregation logic.

Planned enhancements in interactivity, live-data integration, and predictive modelling position the system to grow from a descriptive analytics tool into a forward-looking decision-support platform for cricket analysts and fans alike. More broadly, the methodology and architecture described here are not specific to the IPL and could be adapted with modest effort to any sufficiently granular, publicly available sports dataset, offering a template for future student and independent-analyst projects in the sports analytics space.

From an engineering-education perspective, the project also illustrates a broader point applicable well beyond cricket analytics: a comparatively small set of general-purpose data-science tools — a DataFrame library, a numerical computing library, a plotting library, and a reactive dashboard framework — is sufficient to take a raw, publicly sourced dataset all the way to a polished, interactive, user-facing analytics product, provided that the underlying data-cleaning and aggregation logic is designed with the same rigor typically reserved for production software. The consistency-by-construction property achieved by centralizing every statistic in a single aggregation function, discussed in Sections IV-D and V-B, is presented as a reusable design pattern applicable to dashboard projects well outside the sports domain, including business intelligence and public-health reporting contexts that share the same underlying requirement of translating large, granular event logs into trustworthy, human-readable summaries.

ACKNOWLEDGMENT

The author thanks the maintainers of the public Kaggle IPL ball-by-ball dataset for making high-granularity cricket statistics freely available for analysis, and acknowledges the open-source Streamlit, Pandas, NumPy, and Matplotlib communities whose tools underpin this project.

REFERENCES

- [1] Kaggle, "IPL Ball-by-Ball Dataset," Kaggle Datasets. [Online]. Available: <https://www.kaggle.com/datasets>.
- [2] Streamlit Inc., "Streamlit Documentation," 2024. [Online]. Available: <https://docs.streamlit.io>.
- [3] W. McKinney, "Data Structures for Statistical Computing in Python," in Proc. 9th Python in Science Conf., 2010, pp. 56-61.
- [4] C. R. Harris et al., "Array programming with NumPy," *Nature*, vol. 585, pp. 357-362, 2020.
- [5] J. D. Hunter, "Matplotlib: A 2D Graphics Environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90-95, 2007.
- [6] Board of Control for Cricket in India, "Indian Premier League Official Statistics Archive." [Online]. Available: <https://www.iplt20.com>.
- [7] M. Kimber, "A Statistical Analysis of Twenty20 Cricket," in Proc. Mathematics and Computers in Sport Conf., 2010.
- [8] T. B. Swartz, P. S. Gill, D. Beaudoin, and B. deSilva, "Optimal batting orders in one-day cricket," *Computers & Operations Research*, vol. 33, no. 7, pp. 1939-1950, 2006.
- [9] S. R. Iyer and R. Sharda, "Prediction of athletes performance using neural networks: An application in cricket team selection," *Expert Systems with Applications*, vol. 36, no. 3, pp. 5510-5522, 2009.
- [10] N. Davis, S. R. Perera, and T. B. Swartz, "A simulator for Twenty20 cricket," *Australian & New Zealand Journal of Statistics*, vol. 57, no. 1, pp. 55-71, 2015.
- [11] Plotly Technologies Inc., "Plotly Python Open Source Graphing Library," 2015. [Online]. Available: <https://plotly.com/python>.
- [12] The pandas development team, "pandas-dev/pandas: Pandas," Zenodo, 2020. [Online]. Available: <https://doi.org/10.5281/zenodo.3509134>.